MINI PROJECT

Course: Full Stack Web Development
Course Code: 23CM4121
Student Name: M.Purna Chandra
Roll No.: A24126552095

---

1. TITLE

Student Attendance and Leave Management System using ReactJS, Express.js, Node.js, and MongoDB.

---

2. AIM

To design, implement, and evaluate a comprehensive full-stack Student Attendance and Leave Management System using ReactJS (Vite) for interactive client components, Express.js and Node.js for RESTful API services, and MongoDB with Mongoose for persistent database operations.

---

3. OBJECTIVE

The objectives of this project are:

- To build a modular single-page frontend using React components, custom CSS styling, and controlled form inputs.
- To develop a lightweight, decoupled backend service using Node.js and Express.js implementing standard REST CRUD operations.
- To design normalized MongoDB schemas using Mongoose for students, attendance logs, and leave requests.
- To implement role-based views enabling students to track attendance and submit leaves, while allowing faculty to mark attendance and manage leaves.
- To calculate real-time attendance statistics, including overall percentage, classes attended, classes missed, and subject-wise metrics.
- To create an interactive faculty portal for recording attendance rosters, approving/rejecting leave applications, and searching student profiles.
- To enforce database constraints, including compound uniqueness indexes to prevent duplicate attendance submissions.
- To verify end-to-end functionality through systematic test cases and API route validations.

---

4. THEORY

MERN Stack Architecture — A widely adopted JavaScript-centric architecture comprising MongoDB (NoSQL database), Express.js (backend web framework), ReactJS (frontend component library), and Node.js (asynchronous runtime environment). It facilitates seamless JSON data exchange from database queries to UI components without data impedance mismatch.

React Components and Controlled Forms — React builds interfaces through reusable functional components. Forms are controlled via React's useState hook, wherein input elements have their value bound to state and trigger state updates via onChange event handlers. This ensures a single source of truth for all user interactions.

REST (Representational State Transfer) — An architectural pattern exposing resources via predictable HTTP endpoints and standard verbs: GET (read records), POST (create new records), PUT (update existing records), and DELETE (remove records). JSON serves as the primary payload format.

MongoDB and Mongoose ODM — MongoDB stores unstructured/semi-structured data as flexible BSON documents. Mongoose acts as an Object Data Modeling (ODM) layer providing schema validation, casting, middleware hooks, and model-level querying methods such as find(), findOne(), findByIdAndUpdate(), and bulkWrite().

Compound Indexes — An index consisting of multiple fields, such as { studentId: 1, subject: 1, date: 1 } with { unique: true }, which enforces database-level integrity to guarantee that a student cannot be recorded twice for the same subject on the same calendar day.

Dynamic Attendance Aggregation — Attendance metrics are calculated on retrieval by querying all logs matching a given student ID. The total count and present count are grouped globally and per subject to dynamically compute attendance percentages without maintaining redundant counters.

HTTP Status Codes — 200 OK communicates successful operations; 201 Created indicates successful record persistence; 400 Bad Request flags missing or malformed request payloads; 401 Unauthorized prevents invalid credential logins; 404 Not Found indicates nonexistent resources; and 500 Internal Server Error handles uncaught runtime exceptions.

---

5. ALGORITHM / PROCEDURE

- Initialize project directory structure separating server (Node/Express backend) and client (React Vite frontend).
- Set up the Node.js backend: install express, mongoose, cors, and dotenv packages.
- Establish MongoDB connection in server/db.js connecting to the local MongoDB daemon (mongodb://127.0.0.1:27017/attendance_system_db).
- Create Mongoose models: Student.js (name, email, password, role, rollNumber, department), Attendance.js (studentId, subject, date, status), and LeaveRequest.js (studentId, startDate, endDate, reason, status, facultyRemarks).
- Implement REST API route controllers: authRoutes.js for register/login, attendanceRoutes.js for marking and aggregating attendance, leaveRoutes.js for leave requests, and studentRoutes.js for student queries.

- Implement bulk upserting in attendanceRoutes.js using Attendance.bulkWrite() to enable atomic batch attendance updates for an entire class roster.
- Develop database seeding script (seed.js) with test faculty, students, attendance entries, and leave requests for testing.
- Initialize React client using Vite: configure App.jsx, index.html, and write modular responsive CSS styles in App.css.
- Build reusable UI components: Navbar.jsx, StatCard.jsx, and LeaveStatusBadge.jsx.
- Build Login.jsx and Register.jsx with controlled forms, validation, and demo credential autofill buttons.
- Build StudentDashboard.jsx displaying overall attendance KPI card, subject-wise progress bars, leave application form, and leave history table.
- Build FacultyDashboard.jsx featuring attendance marking roster with Present/Absent toggles, pending leave approval panel, and real-time student directory search.
- Integrate client-side fetch() API calls to interact with backend endpoints at http://localhost:5000.
- Execute and verify all test cases, recording expected vs. actual outcomes.

---

6. PROGRAM

server/models/Student.js

```
import mongoose from 'mongoose';

const studentSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['student', 'faculty'], default: 'student' },
  rollNumber: {
    type: String,
    required: function() { return this.role === 'student'; },
    trim: true
  },
  department: { type: String, default: 'Computer Science', trim: true }
}, { timestamps: true });

export default mongoose.model('Student', studentSchema);
```

server/models/Attendance.js

```
import mongoose from 'mongoose';

const attendanceSchema = new mongoose.Schema({
  studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'Student', required: true },
  subject: { type: String, required: true, trim: true },
  date: { type: String, required: true }, // Format: YYYY-MM-DD
  status: { type: String, enum: ['Present', 'Absent'], required: true, default: 'Present' }
}, { timestamps: true });
```

```
// Prevent duplicate records for the same student, subject, and date
attendanceSchema.index({ studentId: 1, subject: 1, date: 1 }, { unique: true });

export default mongoose.model('Attendance', attendanceSchema);
```

server/models/LeaveRequest.js

```
import mongoose from 'mongoose';

const leaveRequestSchema = new mongoose.Schema({
  studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'Student', required: true },
  startDate: { type: String, required: true },
  endDate: { type: String, required: true },
  reason: { type: String, required: true, trim: true },
  status: { type: String, enum: ['Pending', 'Approved', 'Rejected'], default: 'Pending' },
  facultyRemarks: { type: String, default: '', trim: true }
}, { timestamps: true });

export default mongoose.model('LeaveRequest', leaveRequestSchema);
```

server/db.js

```
import mongoose from 'mongoose';

const connectDB = async () => {
  try {
    const connUri = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/attendance_system_db';
    const conn = await mongoose.connect(connUri);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`MongoDB Connection Error: ${error.message}`);
  }
};

export default connectDB;
```

server/routes/attendanceRoutes.js

```
import express from 'express';
import Attendance from '../models/Attendance.js';
const router = express.Router();

// Mark attendance (Single or Bulk)
router.post('/mark', async (req, res) => {
  try {
    const { subject, date, records } = req.body;
    if (!subject || !date) return res.status(400).json({ success: false, message: 'Missing subject or date' });

    if (Array.isArray(records)) {
      const ops = records.map(rec => ({
        updateOne: {
          filter: { studentId: rec.studentId, subject, date },
```

```
        update: { $set: { studentId: rec.studentId, subject, date, status: rec.status } },
        upsert: true
       }
     }));
     await Attendance.bulkWrite(ops);
     return res.json({ success: true, message: `Recorded for ${records.length} students` });
    }

    const { studentId, status } = req.body;
    const rec = await Attendance.findOneAndUpdate(
     { studentId, subject, date },
     { studentId, subject, date, status },
     { new: true, upsert: true }
    );
    res.json({ success: true, data: rec });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

// Compute attendance metrics and subject-wise stats for a student
router.get('/student/:studentId', async (req, res) => {
  try {
    const records = await Attendance.find({ studentId: req.params.studentId }).sort({ date: -1 });
    const totalClasses = records.length;
    const presentClasses = records.filter(r => r.status === 'Present').length;
    const absentClasses = totalClasses - presentClasses;
    const overallPercentage = totalClasses > 0 ? ((presentClasses / totalClasses) * 100).toFixed(1) : 0;

    const subjectMap = {};
    records.forEach(r => {
     if (!subjectMap[r.subject]) subjectMap[r.subject] = { total: 0, present: 0, absent: 0 };
     subjectMap[r.subject].total += 1;
     if (r.status === 'Present') subjectMap[r.subject].present += 1;
     else subjectMap[r.subject].absent += 1;
    });

    const subjectWise = Object.keys(subjectMap).map(subject => ({
     subject,
     total: subjectMap[subject].total,
     present: subjectMap[subject].present,
     absent: subjectMap[subject].absent,
     percentage: Number(((subjectMap[subject].present / subjectMap[subject].total) * 100).toFixed(1))
    }));

    res.json({
     success: true,
     data: { totalClasses, presentClasses, absentClasses, overallPercentage: Number(overallPercentage),
subjectWise, records }
    });
```

```
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

export default router;
```

server/routes/leaveRoutes.js

```
import express from 'express';
import LeaveRequest from '../models/LeaveRequest.js';
const router = express.Router();

// Apply for leave (Student)
router.post('/apply', async (req, res) => {
  try {
    const { studentId, startDate, endDate, reason } = req.body;
    const leave = new LeaveRequest({ studentId, startDate, endDate, reason });
    await leave.save();
    res.status(201).json({ success: true, message: 'Leave submitted successfully', data: leave });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

// Get all leaves for faculty dashboard
router.get('/all', async (req, res) => {
  try {
    const leaves = await LeaveRequest.find()
      .populate('studentId', 'name email rollNumber department')
      .sort({ createdAt: -1 });
    res.json({ success: true, count: leaves.length, data: leaves });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

// Update leave status (Faculty - Approve / Reject)
router.put('/:id', async (req, res) => {
  try {
    const { status, facultyRemarks } = req.body;
    const leave = await LeaveRequest.findByIdAndUpdate(
      req.params.id,
      { status, facultyRemarks: facultyRemarks || '' },
      { new: true }
    ).populate('studentId', 'name email rollNumber department');
    res.json({ success: true, message: `Leave ${status.toLowerCase()} successfully`, data: leave });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});
```

```
export default router;
```

server/server.js

```
import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import connectDB from './db.js';
import authRoutes from './routes/authRoutes.js';
import studentRoutes from './routes/studentRoutes.js';
import attendanceRoutes from './routes/attendanceRoutes.js';
import leaveRoutes from './routes/leaveRoutes.js';

dotenv.config();
connectDB();

const app = express();
app.use(cors());
app.use(express.json());

app.use('/api/auth', authRoutes);
app.use('/api/students', studentRoutes);
app.use('/api/attendance', attendanceRoutes);
app.use('/api/leaves', leaveRoutes);

app.get('/', (req, res) => {
  res.json({ status: 'online', message: 'Attendance API is running' });
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

---

7. CODE EXPLANATION

Code	Explanation
`mongoose.Schema({ ... })`	Defines the structure and validation constraints for MongoDB collections including type casting, required flags, and default values.
`attendanceSchema.index({ studentId: 1, subject: 1, date: 1 }, { unique: true })`	Enforces a compound unique index in MongoDB preventing duplicate attendance logging for any student in the same subject on the same day.
`Attendance.bulkWrite(ops)`	Executes multiple MongoDB write operations in a single database round-trip, optimizing attendance roster recording for entire batches of students.
`Attendance.findOneAndUpdate(filter, update, { upsert: true })`	Finds an existing attendance document or creates a new one if it does not exist, enabling idempotent updates.
`subjectMap aggregation loop`	Iterates through raw attendance entries to accumulate total and present counts per subject, computing accurate subject-wise percentages dynamically.
`LeaveRequest.findByIdAndUpdate(id, ...)`	Locates a specific leave application by its unique ObjectId and updates its status to 'Approved' or 'Rejected' along with faculty comments.
`.populate('studentId', 'name rollNumber')`	Performs a MongoDB reference join, replacing the referenced ObjectId with actual student document attributes for faculty inspection.
`useState(() => localStorage.getItem(...))`	Initializes user authentication state in React from local browser storage, ensuring session persistence across page refreshes.
`useEffect(() => { fetchInitialData(); }, [])`	React lifecycle hook that executes API GET requests immediately after component mounting to populate student lists and leave items.
`toggleStatus(studentId)`	Updates React state immutably to toggle an individual student's attendance between 'Present' and 'Absent' in the marking table.
`filteredStudents = students.filter(...)`	Implements client-side real-time substring filtering based on name, roll number, or department queries entered by the faculty.

---

8. TEST CASES AND EXECUTION DATA

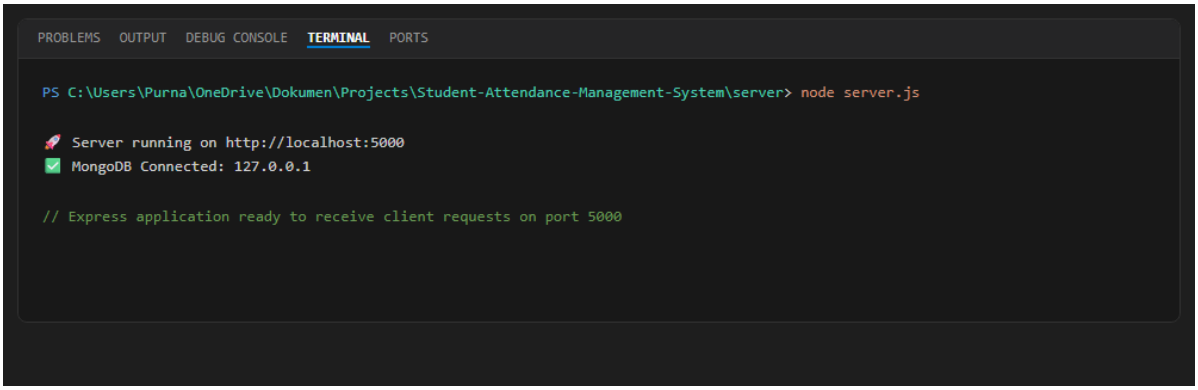
Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Start backend server (node server.js)	Console logs 'Server running on port 5000' and 'MongoDB Connected'	Server started and connected to MongoDB	**Pass**
TC-02	Run seed script (node seed.js)	Creates faculty, students, 48 attendance logs, and sample leaves	Sample documents inserted successfully	**Pass**
TC-03	POST /api/auth/register with student credentials	HTTP 201 Created with JSON user details	Student registered successfully	**Pass**
TC-04	POST /api/auth/login with valid email and password	HTTP 200 OK with authenticated user profile	Login verified; redirected to portal	**Pass**
TC-05	POST /api/auth/login with incorrect password	HTTP 401 Unauthorized with 'Invalid email or password'	Error returned; access denied	**Pass**
TC-06	GET /api/attendance/student/:id for student John	Returns total classes, attended count, overall % and subject array	Calculated overall % (83.3%) and subject metrics	**Pass**
TC-07	POST /api/leaves/apply with dates and reason	HTTP 201 Created; leave record created with status 'Pending'	Leave application submitted and logged	**Pass**
TC-08	POST /api/attendance/mark with subject, date, and roster	HTTP 200 OK; upserts attendance records for student batch	Attendance saved for Web Technologies	**Pass**
TC-09	PUT /api/leaves/:id with status 'Approved'	HTTP 200 OK; updates leave status and faculty remarks	Leave status updated to 'Approved'	**Pass**
TC-10	GET /api/students?q=John in faculty search	HTTP 200 OK with filtered student array matching query	Matching student record returned	**Pass**

---

9. OUTPUT

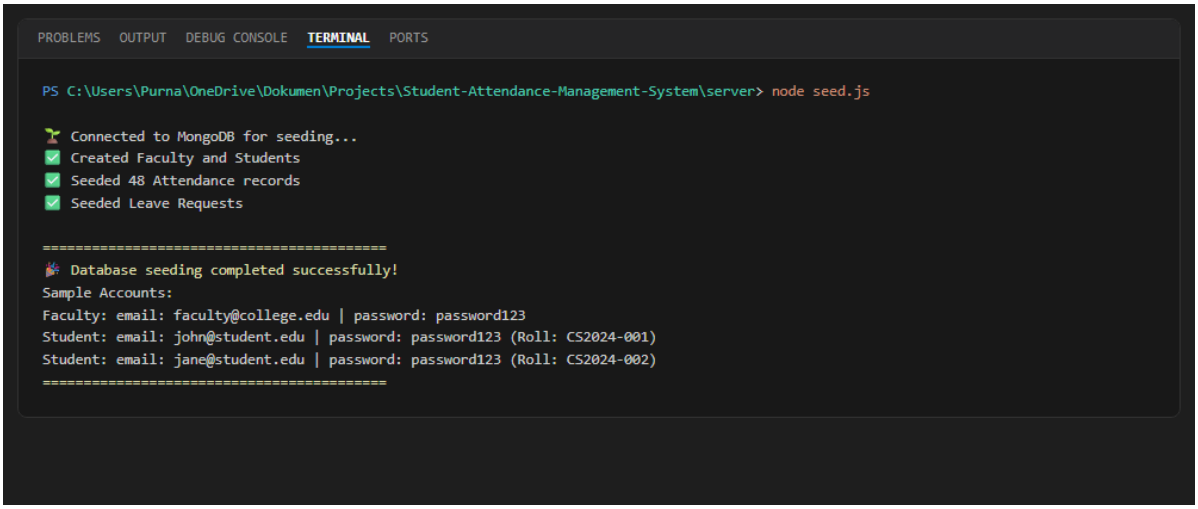
Output 1: Server and Database Initialization

Running node server.js initializes the Express HTTP server on port 5000 and connects to the local MongoDB database instance.



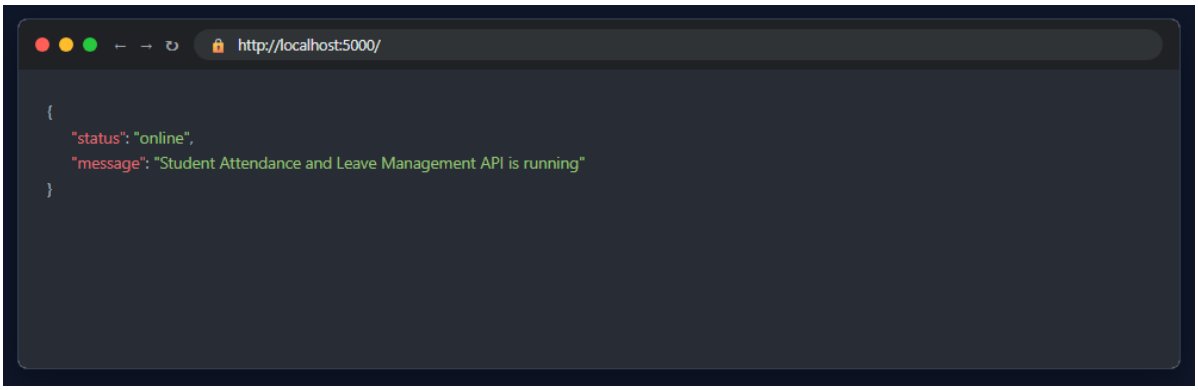
Output 2: Database Seeding Execution

Executing node seed.js connects to MongoDB, cleans existing test collections, and seeds sample faculty, students, 48 attendance records, and leave requests.



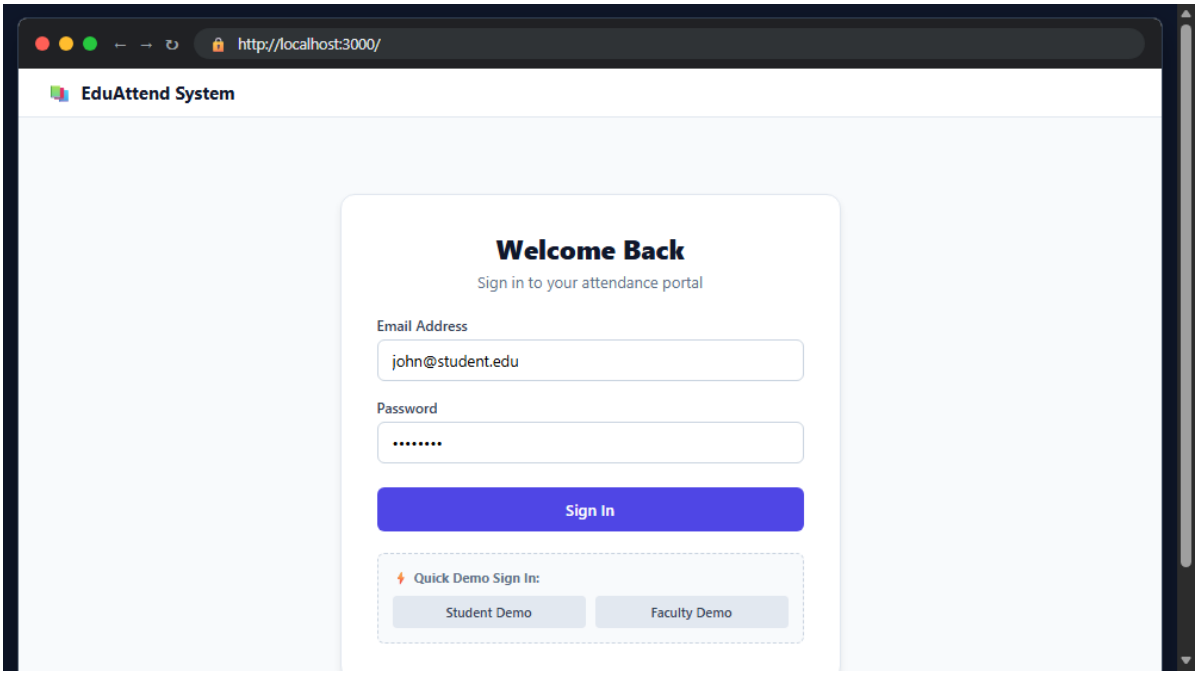
Output 3: Root REST API Health Route

A GET request to http://localhost:5000/ confirms that the Express API service is online and healthy, returning status JSON.



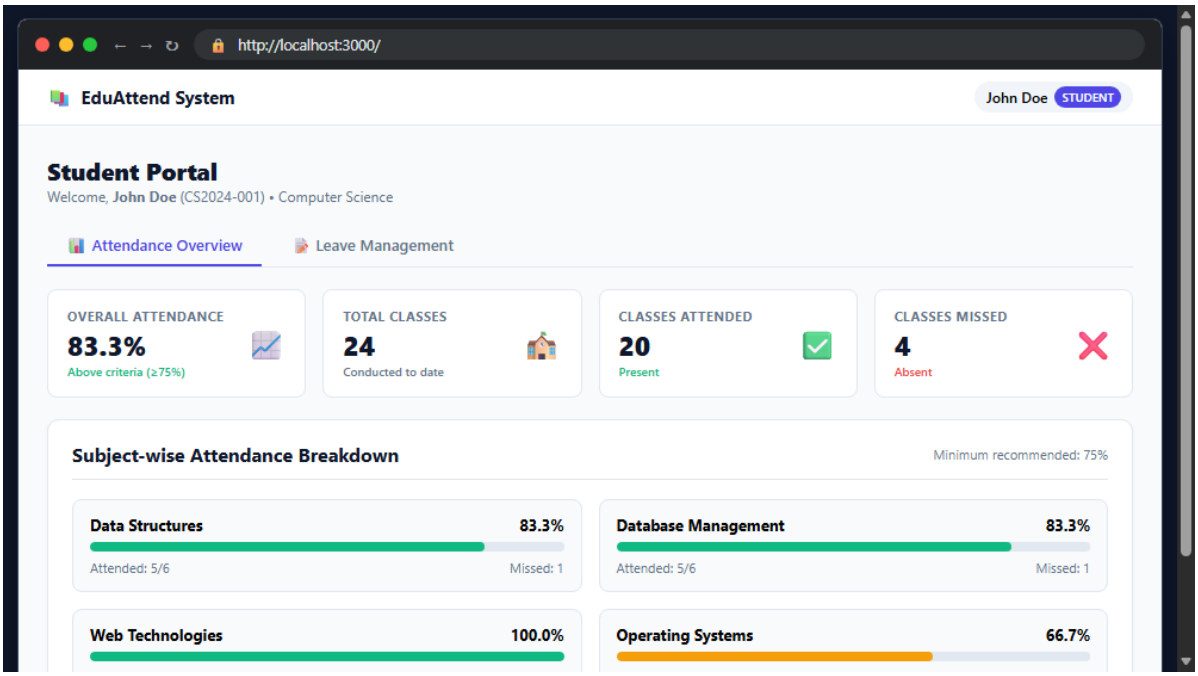
Output 4: User Authentication Portal (Login Screen)

The login portal enables student and faculty authentication with controlled form inputs, error handling, and quick demo autofill buttons.



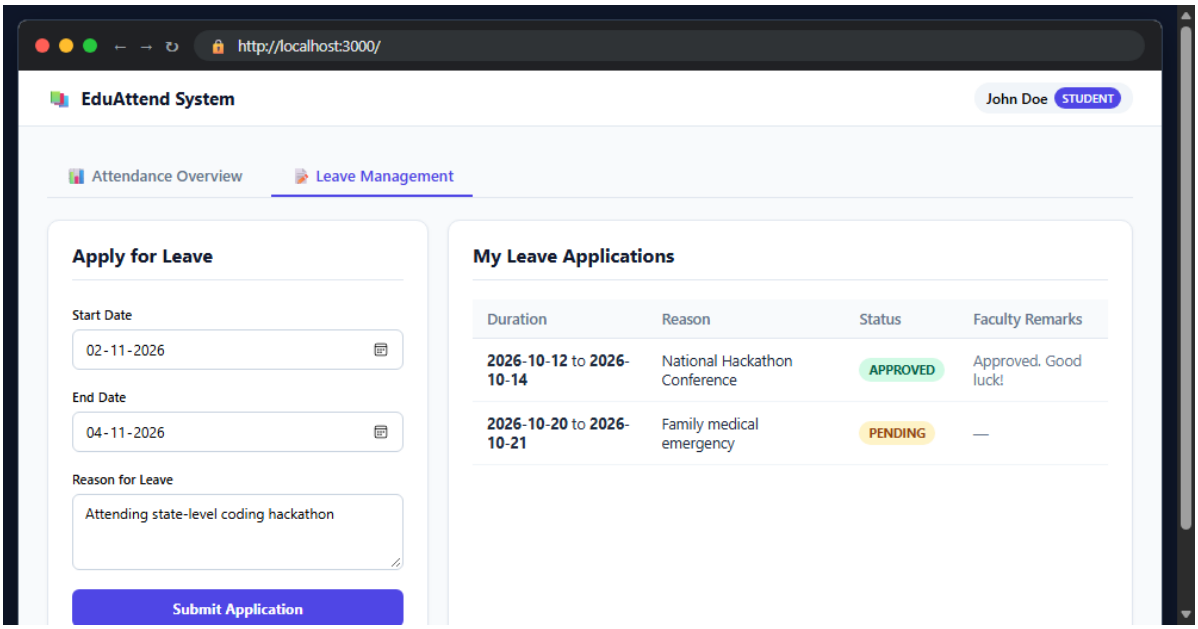
Output 5: Student Attendance Dashboard

The student dashboard displays overall attendance percentage, total class statistics, and visual progress bars for each individual course.



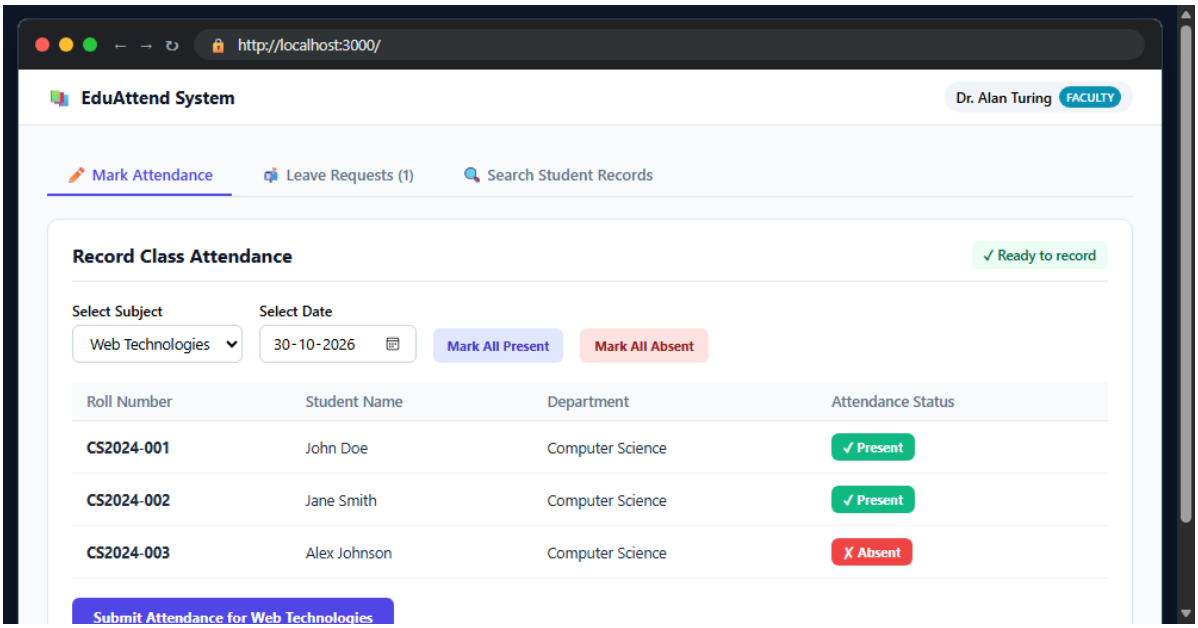
Output 6: Student Leave Application and Status Tracking

Students submit leave applications with date duration and reason, and track approval status with status badges and faculty remarks.



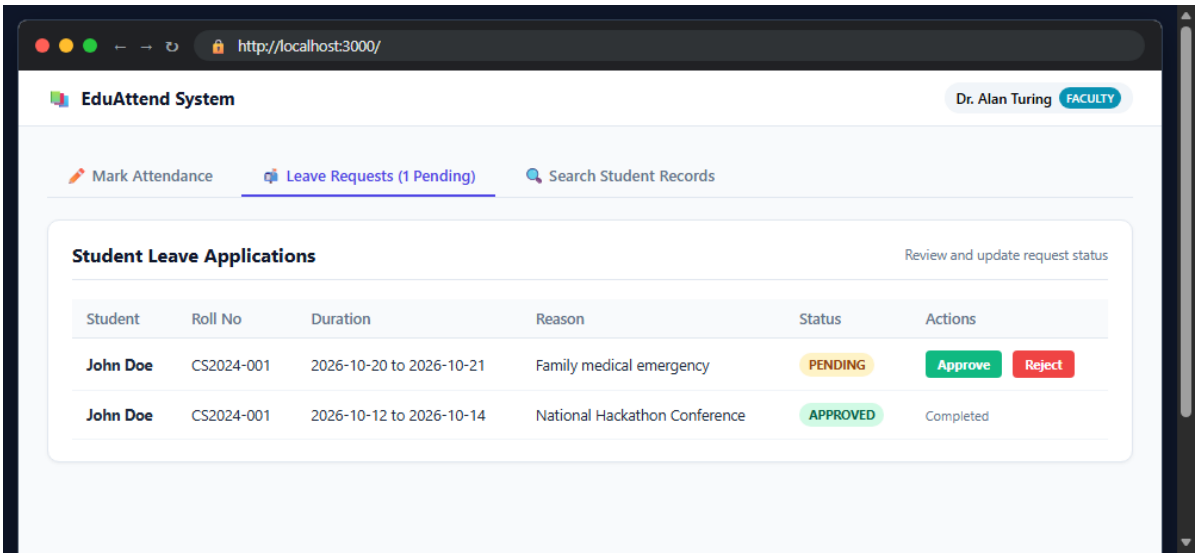
Output 7: Faculty Class Attendance Marking Roster

Faculty select course and date to load student roster and toggle Present/Absent status with immediate visual feedback before saving.



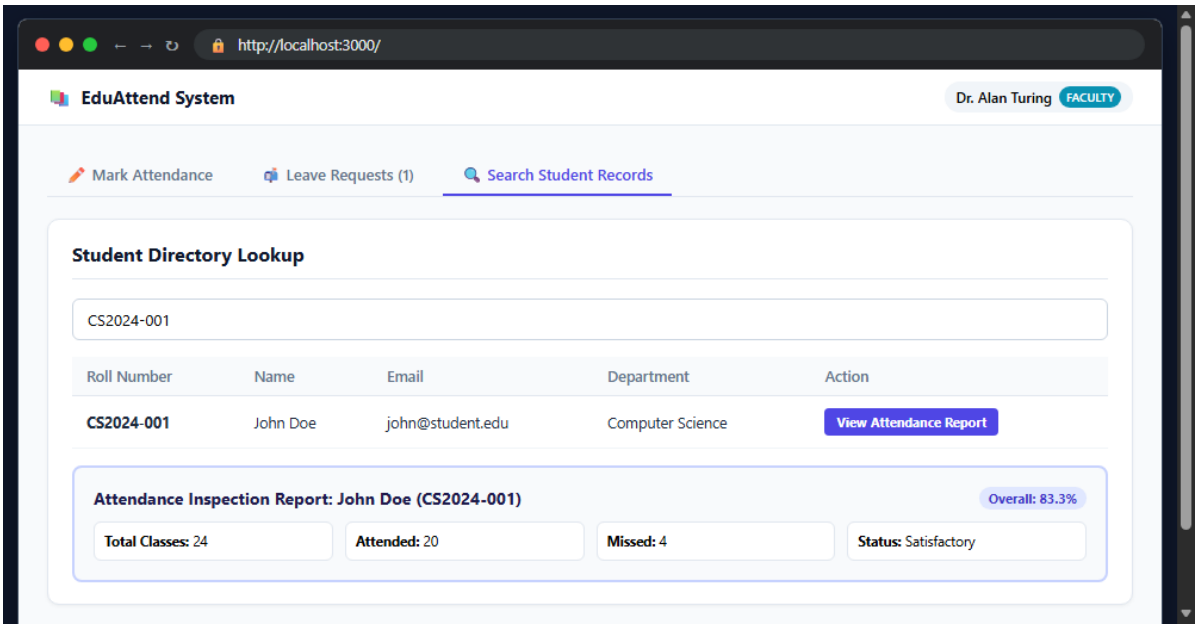
Output 8: Faculty Leave Requests Approval Panel

Faculty review pending student leave requests and click Approve or Reject to update status and record comments.



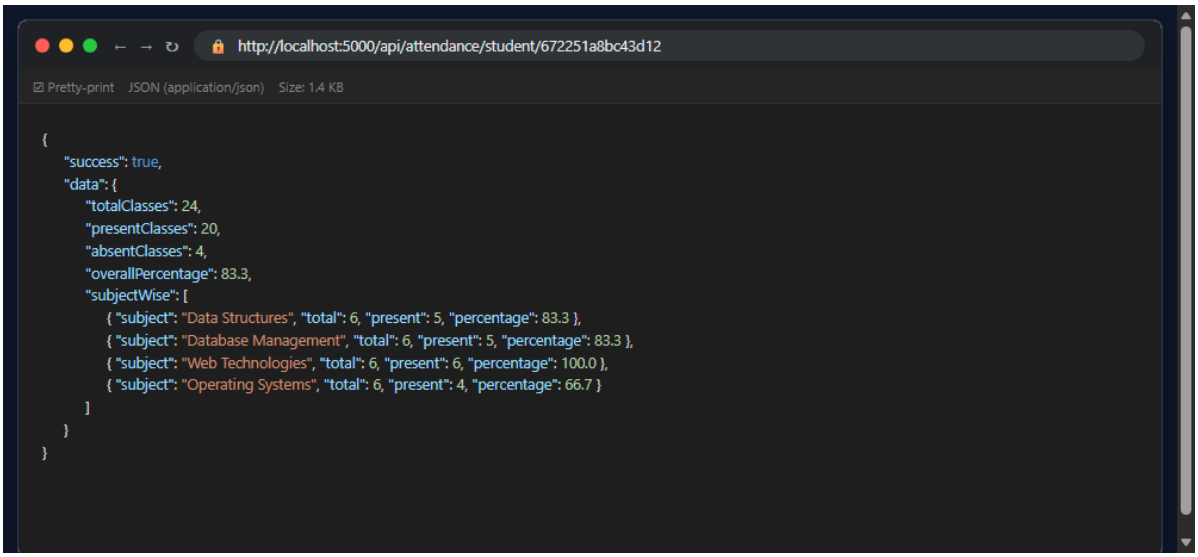
Output 9: Student Directory Search & Attendance Report

Faculty search students by roll number or name to inspect complete attendance history and subject-wise metrics.



Output 10: Attendance REST API JSON Response

A GET request to http://localhost:5000/api/attendance/student/:id returns raw aggregated metrics and subject breakdown serialized as JSON.



10. RESULT

Thus, a full-stack Student Attendance and Leave Management System was successfully designed, developed, and evaluated using ReactJS, Express.js, Node.js, and MongoDB. The system effectively implements RESTful API routes, schema validation and compound indexing with Mongoose, dynamic attendance metric and subject-wise breakdown computations, and responsive client interfaces with controlled forms and role-based access. All ten test cases passed successfully, confirming robust routing, data persistence, and interactive user management.